Linux System Administration

Mid-Term Examination: Mini Project Instructions

Practical Assessment: Web Services, Automation and Documentation

Instructor	Joshua K. IRADUKUNDA
Assessment Type	Practical Mini Project (Mid-Term Examination)
Submission Deadline	Saturday, 24 October 2026 at 11:59 PM
Submission Format	Three recorded video tutorials (one video per task)
Work Mode	Individual work
Total Marks	100

AT A GLANCE

Task	What you build	Points	Your video
Task 1: Apache2 website	A secured website served over HTTP and HTTPS with your own self-signed certificate, tested from a second device.	30	Video 1
Task 2: Backup automation	A report script, a cron job that transfers new backups every minute, and an email notification when a backup arrives.	35	Video 2
Task 3: NGINX documentation	A tutorial-style documentation site, served by NGINX, that explains how you completed Tasks 1 and 2.	20	Video 3
Video and ID compliance	Three complete, correctly named videos that show your Student ID wherever it is required.	15	Videos 1 to 3

How to use this document

- Read the whole document once before you start. Then follow the Recommended Order of Work in Section 1.
- **12345** is an example Student ID. Replace it with your own Student ID everywhere.
- Text in angle brackets, such as <VM1 IP>, is a placeholder that you must replace with your own value.

Table of Contents

1. Overview **3**
Recommended Order of Work 3
2. Learning Outcomes Assessed **4**
3. General Rules **4**
4. Before You Start: Required Setup **4**
Lab Layout 5
Network Settings That Prevent Most Problems 6
Your Personal Values Worksheet 6
5. Task 1: Secure Web Server Deployment with Apache2 (30 points) **7**
Your Identity Values 7
Part A: Prepare the Linux Web Server 7
Part B: Create Your Website 8
Part C: Configure the HTTP Virtual Host 9
Part D: Configure Your Student Domain 9
Part E: Generate Your Own Self-Signed SSL/TLS Certificate 10
Part F: Configure HTTPS 12
Part G: Configure the Firewall 12
Part H: Test Your Website Locally 13
Part I: Test from a Second Device 13
6. Task 2: Automated Cross-System Backup with Email Notification (35 points) **15**
Task 2 Setup: SSH and Folders 15
Part A: The Report Script (Runs Manually, on VM1) 16
Part B: The Auto-Backup Cron Job (Runs Automatically, on VM1) 16
Part C: Arrival Notification (on VM2) 17
Rehearse the Whole Chain Before Recording 18
7. Task 3: Documentation Server with NGINX (20 points) **19**
Task 3 Requirements 19
Where to Run NGINX 19
Deploy NGINX 19
What Your Documentation Must Contain 19
8. Submission Requirements **20**
Every Video Must 20
Recording Tips 21
File Naming and Submission 21
9. Academic Integrity and Student ID Policy **21**
11. Evaluation Criteria **22**
12. Final Checklist Before You Submit **23**

1. Overview

This mid-term examination replaces a written test with a hands-on mini project. It evaluates your ability to independently deploy, secure, automate, and document real Linux server infrastructure, the same core responsibilities expected of a junior Linux system administrator on the job.

You will complete three integrated tasks: deploying a secured web server, building an automated backup pipeline between two systems, and producing professional technical documentation of your own work. You will submit video evidence for each task rather than a written report.

> **Important**
> This project is designed so that copied or shared work is easy to detect. Read Section 9 (Academic Integrity and Student ID Policy) carefully before you begin.

Recommended Order of Work

1. **Prepare your environment (Section 4).** Two machines, a working network, the required tools, and a second device for browser tests.
2. **Task 1, Parts A to D.** Create your website, the HTTP virtual host, and your domain name. Confirm that the site loads by name.
3. **Task 1, Parts E to H.** Generate and verify your certificate, configure HTTPS and the firewall, and test locally.
4. **Task 1, Part I.** Test from the second device. Then record **Video 1**.
5. **Task 2.** Set up SSH, write the report script, create the cron transfer, and configure the email notification. Rehearse the whole chain, then record **Video 2**.
6. **Task 3.** Build the NGINX documentation site from the notes you kept during Tasks 1 and 2. Then record **Video 3**.
7. **Submit.** Work through the Final Checklist (Section 12), name your files correctly, and upload before the deadline.

> **Tip**
> Keep a notes file from the first minute. Paste every important command and every configuration change into it as you work. These notes become your Task 3 documentation and save you hours.
>
> Plan to finish the configuration well before the deadline. A video that has to be recorded again costs more time than the configuration did.

2. Learning Outcomes Assessed

Outcome	Assessed in
Install and configure the Apache2 and NGINX web servers on Linux	Tasks 1 and 3
Generate and apply a self-signed SSL/TLS certificate to enable HTTPS	Task 1
Apply Linux user, file, and permission concepts in a real deployment	Tasks 1 and 2
Write and run Bash scripts that collect live system information	Task 2
Schedule recurring tasks using cron	Task 2
Configure secure inter-system file transfer and an automated backup routine	Task 2
Configure basic outbound email notification from a Linux system	Task 2
Produce clear, reproducible technical documentation	Task 3

3. General Rules

1. **Individual work.** Every student must build and demonstrate their own deployment. Do not share scripts, configuration files, certificates, or recordings with classmates.
2. **Same environment as class.** Use the same environment and tools demonstrated in class (same distribution family and the same configuration approach) unless your instructor has told you otherwise. This keeps submissions comparable and makes copied work obvious. The commands in this document use the Debian and Ubuntu family (`apt`, `apache2`, `a2ensite`, `ufw`). If your class used another distribution, use its equivalent commands.
3. **Student ID everywhere.** Your Student ID must appear throughout your work, not just on a cover page. Section 9 lists every place where it is required.
4. **Same LAN.** All systems used for Tasks 1 to 3 must be on the same LAN (local network), so that connectivity between them, and access from a second device, can be demonstrated live.
5. **Physical or virtual machines.** You may use physical machines or virtual machines (VirtualBox, VMware, or similar). What matters is that the final result is reachable over the network, not how the machine was provisioned.

4. Before You Start: Required Setup

Confirm that you have the following before you start recording. None of this is new: it is exactly what was covered and installed in class.

Requirement	Details	Quick check
Two Linux machines or VMs	Same LAN and subnet, able to ping each other. They can be two VMs on one host or two separate machines.	`ping -c 3 <other IP>` from each machine
A second device	A laptop or PC on the same LAN, used for browser tests. You must be able to edit its hosts file (Task 1, Part D).	Ping the IP address of VM1 from this device
Sudo or root access	On both systems, for package installation and service configuration.	`sudo -v`

Requirement	Details	Quick check
Apache2	`apt install apache2` (or your distribution's equivalent). Needed on VM1.	`apache2 -v`
NGINX	`apt install nginx` (or your distribution's equivalent). Needed for Task 3.	`nginx -v`
OpenSSL	Generates your self-signed certificate. Pre-installed on almost all distributions. Version 1.1.1 or newer is recommended.	`openssl version`
SSH client and server	`openssh-server` on both VMs, for secure, automatable file transfer.	`ssh -V` and `systemctl is-active ssh`
A mail utility	For example `mailutils` with `postfix`, `msmtp`, or `ssmtp`, or any tool of your choice that you can configure to send outbound email. Needed on VM2.	Send a test message after you configure it (Task 2, Part C)
zip and unzip	For packaging the backup files in Task 2. Needed on VM1.	`which zip unzip`
Screen recording software	OBS Studio, SimpleScreenRecorder, any online meeting platform, or any tool that records your screen and audio.	Record 30 seconds and play it back with sound

Lab Layout

Use the layout in Figure 1. It keeps all three tasks on two VMs plus one browser device, and it avoids port conflicts.

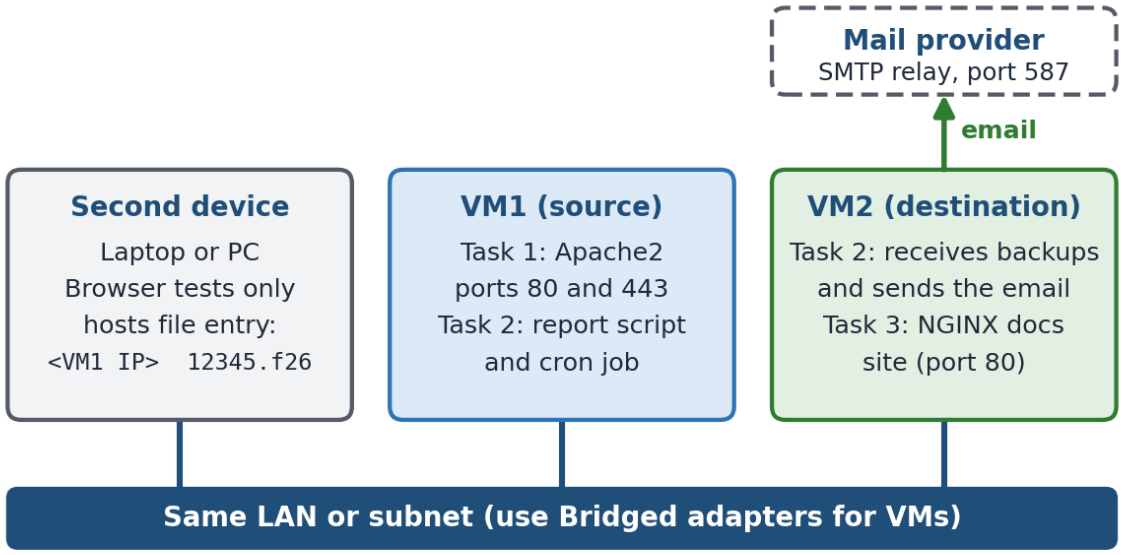


Figure 1: Recommended lab layout for Tasks 1 to 3

- **VM1 (source):** hosts the Apache2 website (Task 1) and creates and sends the backup files (Task 2).
- **VM2 (destination):** receives the backup files and sends the email notification (Task 2). It can also host the NGINX documentation site (Task 3) on the default port 80, because Apache2 is not installed there.
- **Second device:** a laptop or PC with a browser, used to test Tasks 1 and 3 over the LAN.
- If you prefer to run NGINX on VM1, use ports other than 80 and 443 (for example 8080 and 8443), because Apache2 already uses them.

Network Settings That Prevent Most Problems

- In VirtualBox or VMware, set each VM's network adapter to **Bridged**. In NAT mode, other devices on your LAN cannot reach the VM.
- Find each VM's IP address with `hostname -I` (or `ip a`) and write it in the worksheet below.
- Some campus and public Wi-Fi networks isolate devices from each other, so pings fail even though the devices appear to share a network. If this happens, use another network, such as a personal router or a phone hotspot.
- IP addresses assigned by DHCP can change after a reboot. Check them again before you record, and update your hosts entries if they changed.

Your Personal Values Worksheet

Fill this in before you begin and keep it open while you work. Every command and file in this document uses these values.

Item	Example	Your value
Full name	Jane Doe	
Student ID	12345	
Domain (<StudentID>.f26)	12345.f26	
VM1 IP address	192.168.1.50	
VM2 IP address	192.168.1.51	
Email address that receives notifications	jane@example.com	

5. Task 1: Secure Web Server Deployment with Apache2 (30 points)

In this task you configure a Linux machine or VM to host your own website using **Apache2**. The website must be reachable through both **HTTP and HTTPS**. You create and configure your own **self-signed SSL/TLS certificate using OpenSSL**, and you verify the result from a second device connected to the same LAN.

You are expected to do the configuration independently and to explain it in your video. The task assesses: Linux system administration, Apache2 configuration, Apache virtual hosts, HTTP and HTTPS, SSL/TLS certificates, OpenSSL, Linux users and file permissions, firewall and network configuration, HTML, CSS, and JavaScript, and basic troubleshooting and service verification.

Your Identity Values

Your configuration must be based on **your own Student ID**. Use the following information throughout the task.

Item	Required value	Example (Student ID 12345)
Institution	**AUCA**	AUCA
Course	**Linux-F26**	Linux-F26
Owner	**Your Student ID**	12345
Domain	<StudentID>.f26	12345.f26
Certificate type	**Self-signed SSL/TLS certificate**	Self-signed
Certificate validity	**1 year** (about 365 days)	365 days
Website addresses	http://<StudentID>.f26 and https://<StudentID>.f26	http://12345.f26 and https://12345.f26

> **Important**
> The example Student ID 12345 is for illustration only. Replace it with your own Student ID throughout the entire assignment. Your Student ID must be consistent across your website, your HTTP virtual host, your HTTPS virtual host, your SSL/TLS certificate, your domain name, and your demonstration.

Part A: Prepare the Linux Web Server

Goal: Install Apache2 and confirm that it runs and answers on VM1.

WHAT TO DO

- Update the package list, then install `apache2` and `openssl` on VM1.
- Confirm that the Apache2 service is running and enabled at boot.
- Open the default Apache page in a browser on VM1 to confirm that the server answers.

CHECK IT

```
systemctl is-active apache2
systemctl is-enabled apache2
```

The two commands should print `active` and `enabled`.

BE READY TO EXPLAIN

- The role of Apache as a web server, and where Apache obtains the website content that it serves.

Part B: Create Your Website

Goal: Build a dedicated, professional website that summarizes what you have learned so far in the Linux course.

WHAT TO DO

1. Create a dedicated folder for the site in an appropriate Apache web directory, for example `/var/www/12345.f26`.
2. Build the site with **HTML, CSS, and JavaScript**. A clean structure is `index.html`, one style sheet, and one script file.
3. Set ownership and permissions so that Apache can read the files (files readable, directories searchable) and other users cannot modify them.
4. Show your **Student ID** clearly on the page (for example in the header or footer), together with AUCA and Linux-F26.
5. Write the required content described below.

DESIGN EXPECTATIONS

The website must be creative, well structured, readable, and professional. A plain page containing only unformatted text is not sufficient. Use HTML for structure, CSS for styling, and JavaScript for something visible and useful (for example a navigation menu, a search or filter box, or a short quiz). You are encouraged to show your front-end skills through page layout, navigation, typography, colors, sections, cards, tables, interactive elements, responsive design, and JavaScript functionality. The design should support the content rather than distract from it.

REQUIRED WEBSITE CONTENT

Your webpage must contain a meaningful written summary of what you have learned so far in the Linux course. Your discussion must cover **all three** of the following areas.

Topic	Your page must explain
1. Linux Basic Commands	Explain the Linux commands and concepts you have learned, and show how Linux is used to: • Navigate the file system • Create and manage files and directories • View and manipulate files • Search for information • Monitor or interact with the system • Manage services • Obtain basic system and network information Do not simply provide a list of commands. For selected commands, explain **what** the command does, **why** it is useful, and **a practical situation** in which you would use it. *Examples you may choose from (you may use others):* `pwd`, `ls`, `cd`, `mkdir`, `cp`, `mv`, `cat`, `less`, `grep`, `find`, `ps`, `top`, `df`, `systemctl`, `ip`, `ping`.
2. Shell Scripting	Explain the purpose of shell scripting and what you have learned about writing and executing shell scripts. You may include concepts such as variables, input and output, conditions, loops, functions, command execution, exit status, script permissions, and automation. Where appropriate, include examples from your own learning experience. Show that you understand how shell scripting automates Linux administration tasks.
3. Linux Users and File Permissions	Explain how Linux manages users, groups, ownership, and permissions. Show that you understand: • Users, groups, and file ownership • Read, write, and execute permissions • Symbolic and numeric permissions • Changing permissions and changing ownership Explain why Linux file permissions are important for security and system administration.

> **Quality rules for your page**
> - Write in your own words, based on your own learning.
> - Make the page work without internet access: keep fonts, images, and scripts as local files instead of linking to external websites.

CHECK IT

```
ls -l /var/www/12345.f26
```

You should see your files with sensible ownership and permissions (for example mode 644 for files and 755 for directories).

Part C: Configure the HTTP Virtual Host

Goal: Serve your website through a dedicated virtual host on HTTP port 80, identified by your domain.

WHAT TO DO

1. In Apache's `sites-available` directory, create a separate configuration file for the HTTP virtual host (for example `12345.f26.conf`).
2. Configure it to listen on **port 80**, to use your assigned domain as its server name, and to point to the folder that contains your own website.
3. Enable the site, test the configuration syntax, and reload Apache.

KEY DIRECTIVES AND COMMANDS

Directives: `<VirtualHost *:80>`, `ServerName`, `DocumentRoot`, `ErrorLog`, `CustomLog`. Commands: `a2ensite`, `apache2ctl configtest`, `systemctl reload apache2`.

> **Note**
> You cannot test by name until the domain resolves to your server. Finish Part D, then test.

BE READY TO EXPLAIN

- The purpose of a virtual host, and how Apache determines which website to serve when a particular domain is requested.

Part D: Configure Your Student Domain

Goal: Make `<StudentID>.f26` resolve to the IP address of your web server, on VM1 and on the second device.

Your domain is a **local assignment domain**. It does not need to be registered as a public Internet domain, and it does not exist in public DNS. You are responsible for making it resolve to the IP address of your Linux machine or VM when you test within your LAN. A hosts file entry is the simplest method.

WHAT TO DO

1. Find the IP address of VM1 with `hostname -I`.
2. On VM1, add one line to `/etc/hosts`: the IP address, a space, then your domain. Example: `192.168.1.50 12345.f26`
3. On the **second device**, add the same line to its hosts file. On Windows the file is `C:\Windows\System32\drivers\etc\hosts` (open Notepad as Administrator). On Linux and macOS it is `/etc/hosts` (edit it with `sudo`).
4. On both machines, test the name with `ping`. On Linux and macOS use `ping -c 2 12345.f26`. On Windows use `ping 12345.f26`. The reply must come from the IP address of VM1.

> **Important**
> The second device used for testing must also resolve your StudentID-based domain to the correct Linux server. Without the hosts entry on the second device, Part I will fail.

Tip

Test names with `ping` or `getent hosts <domain>`. Do not use `nslookup` or `dig`: they ask DNS servers and ignore the hosts file, so they report that the name does not exist even when your hosts entry is correct.

Phones and tablets usually cannot edit a hosts file. Use a laptop or PC as your second device.

Checkpoint 1

From VM1, `curl -I http://12345.f26` returns `HTTP/1.1 200 OK` and `ping 12345.f26` replies from the IP address of VM1. Do not continue until both are true.

Part E: Generate Your Own Self-Signed SSL/TLS Certificate

This part is **mandatory**. You must generate the certificate yourself with **OpenSSL on your own Linux machine or VM**, and it must be **self-signed**. The purpose is to show that you understand how certificates are generated and deployed in a Linux environment.

Not allowed

Do not use any of the following:

- Another student's certificate
- Another student's private key
- A certificate downloaded from another source
- An online certificate generator
- A publicly issued certificate instead of your required self-signed certificate

REQUIRED CERTIFICATE INFORMATION

Requirement	Certificate field	Required value	Example (12345)
Institution	Organization (O)	**AUCA**	AUCA
Course	Organizational Unit (OU)	**Linux-F26**	Linux-F26
Owner	Common Name (CN)	**Your Student ID**	12345
Domain	Subject Alternative Name (SAN), DNS entry	`<StudentID>.f26`	12345.f26
Validity	Validity period	**1 year** (about 365 days from the day you generate it)	365 days
Type	Issuer and Subject	Identical, because the certificate is self-signed	Issuer equals Subject

> **Note**
> The Organization field must be AUCA, so your Student ID goes in the Common Name (CN). Browsers compare the website name with the SAN, not the CN, so the SAN is what makes https://12345.f26 match your certificate.

PRIVATE KEY AND CERTIFICATE

Generating the certificate creates two different security objects. Understand the purpose of each.

Object	Purpose	Rules
Private key	Proves that your server really owns the certificate. It must stay confidential.	• Keep it on your Linux machine. • Protect it with Linux file permissions (readable only by root). • Never share it with another student, never publish it on your website, and never upload it to a public repository. • Never show its contents in your video.
Certificate	Contains the information that lets the server and clients identify the SSL/TLS service. It is not secret.	It may be readable by the appropriate system services, but it must correspond to the private key that Apache uses.

WHAT TO DO

1. Generate the private key and the self-signed certificate with **one OpenSSL command** (the `req` subcommand). The options you need are listed in the box below this list.
2. The `-subj` option takes the subject in the form `/O=<organization>/OU=<unit>/CN=<name>`. The SAN is passed to `-addext` in the form `subjectAltName=DNS:<domain>`.
3. Store the key in `/etc/ssl/private` and the certificate in `/etc/ssl/certs` (standard locations on Debian and Ubuntu), and name both files after your domain.
4. Set the key's permissions so that only root can read it (mode `600`).

OPTIONS TO LOOK UP (CLASS NOTES OR THE MANUAL PAGE)

```
Manual page : man openssl-req
Subcommand  : req
Options     : -x509  -newkey  -nodes  -keyout  -out  -days  -subj  -addext
```

> **Tip**
> `-nodes` creates a key without a passphrase, so Apache can start without asking you to type one. `-addext` needs OpenSSL 1.1.1 or newer (check with `openssl version`).

VERIFY BEFORE YOU CONTINUE

Inspect the certificate before you configure Apache. Do not proceed to the HTTPS configuration if the certificate contains incorrect student information or an incorrect domain. Regenerate the certificate if necessary.

```
openssl x509 -in /etc/ssl/certs/12345.f26.crt -noout -subject -issuer -dates
openssl x509 -in /etc/ssl/certs/12345.f26.crt -noout -ext subjectAltName
openssl x509 -in /etc/ssl/certs/12345.f26.crt -noout -text
```

Confirm	Where to look	Expected (Student ID 12345)
Organization	subject	O=AUCA
Course (Organizational Unit)	subject	OU=Linux-F26
Student ID	subject	CN=12345
Subject Alternative Name	subjectAltName	DNS:12345.f26
Issuer	issuer	Same as the subject (self-signed)
Validity dates	notBefore and notAfter	One year (365 days) apart
Public-key information	-text output	For example rsaEncryption, 2048 bit

The layout of the output differs between OpenSSL versions. Compare the values, not the spacing.

Part F: Configure HTTPS

Goal: Serve the same website over HTTPS with the certificate you generated.

WHAT TO DO

1. Enable Apache's SSL module (a2enmod ssl).
2. Create a **second, separate** configuration file for the HTTPS virtual host (for example 12345.f26-ssl.conf). It must be configured independently from the HTTP virtual host.
3. The HTTPS virtual host must listen on **port 443**, use your StudentID-based domain, serve the **same website folder** as the HTTP virtual host, use **your own certificate**, and use the **corresponding private key**.
4. Enable the site, test the configuration syntax, and restart Apache.

KEY DIRECTIVES

<VirtualHost *:443>, ServerName, DocumentRoot, SSLEngine on, SSLCertificateFile, SSLCertificateKeyFile.

> **Tip**
> After you enable a module, use a full restart (systemctl restart apache2). For later configuration changes, a reload is enough.

Part G: Configure the Firewall

Goal: Allow web traffic through the firewall, and keep the firewall enabled.

WHAT TO DO

1. Determine whether your Linux machine has an active firewall: sudo ufw status.
2. If the firewall is **active**, allow incoming TCP port **80** (HTTP) and TCP port **443** (HTTPS). If you also reach this machine over SSH, keep port 22 allowed.
3. If the firewall is **inactive**, show the output in your video and state that no firewall rule is needed.

KEY COMMANDS

ufw allow 80/tcp, ufw allow 443/tcp, ufw status verbose. If your distribution uses another firewall tool, use its equivalent.

> **Warning**
> Do not disable the firewall simply to make the assignment work. The objective is to show that you understand how a firewall controls network access to services.

BE READY TO EXPLAIN

- Why port 80 is required.

- Why port 443 is required.
- Why allowing the appropriate ports is necessary for another device to access your server.

Part H: Test Your Website Locally

Before you test from another device, verify the configuration on the Linux machine itself. Do not assume that the configuration is correct simply because Apache starts. Verify that the correct virtual hosts and certificates are actually being used.

Check	Command	Expected result
Apache is running	`systemctl is-active apache2`	`active`
Apache accepts the configuration	`sudo apache2ctl configtest`	`Syntax OK`
HTTP and HTTPS virtual hosts are enabled	`sudo apache2ctl -S`	Lines for `*:80` and `*:443` that show your domain
Ports 80 and 443 are available	`sudo ss -tlnp | grep -E ':80|:443'`	`apache2` listening on both ports
Website files are accessible over HTTP	`curl -I http://12345.f26`	`HTTP/1.1 200 OK`
Website is accessible over HTTPS	`curl -kI https://12345.f26`	`HTTP/1.1 200 OK` (`-k` accepts the self-signed certificate)
Private key has appropriate permissions	`ls -l /etc/ssl/private/12345.f26.key`	Mode 600 (owner read and write only), owned by root
Something is wrong	`sudo tail -n 20 /var/log/apache2/error.log`	Read the newest error lines

TWO EXTRA CHECKS

Confirm that Apache serves **your** certificate, and that the certificate and the private key belong together.

```
echo | openssl s_client -connect 12345.f26:443 -servername 12345.f26 2>/dev/null | openssl x509 -noout -subject -dates

openssl x509 -in /etc/ssl/certs/12345.f26.crt -noout -pubkey | openssl sha256
sudo openssl pkey -in /etc/ssl/private/12345.f26.key -pubout | openssl sha256
```

The first command must show your own subject and dates. The last two commands must print identical hashes.

> **Checkpoint 2**
> Every check above passes, and `apache2ctl -S` shows both virtual hosts for your domain.

Part I: Test from a Second Device

This is a **mandatory demonstration requirement**. Use a second computer, laptop, or other suitable device connected to the **same LAN** as your Linux machine or VM.

WHAT TO DO

1. Confirm that the hosts entry from Part D exists on the second device.
2. In the browser, type the **full address**, including `http://` or `https://`. Without it, the browser may treat the name as a search query.
3. Open `http://<StudentID>.f26`. Your website must load.
4. Open `https://<StudentID>.f26`. The browser shows a warning (see below). Choose Advanced, then proceed to the site. Your website must load over HTTPS.
5. Open the certificate details in the browser and show the values listed in the table below.

Both addresses must reach your website, and the HTTP and HTTPS versions must display the same website.

> **About the browser warning**
> Because your certificate is self-signed, the browser normally warns that the connection is not private, that the certificate is not trusted, or that it was issued by an unknown authority. This warning is **expected** for this assignment and does **not** automatically mean that your SSL/TLS configuration is wrong. Accept the warning, proceed, and show that the website loads over HTTPS. You are **not** required to remove the warning, and you must not replace your self-signed certificate with a publicly trusted certificate to remove it.

CERTIFICATE DETAILS THE LECTURER MUST BE ABLE TO VERIFY

Certificate requirement	Expected
Institution	AUCA
Course	Linux-F26
Student ID	Your Student ID
Domain / SAN	<StudentID>.f26, matching the address in the address bar
Validity	Approximately 1 year
Certificate type	Self-signed

> **Tip**
> If your browser forces https:// or blocks plain HTTP, turn off that setting for the demonstration (for example Chrome's Always use secure connections option or Firefox's HTTPS-Only Mode), or use a private window. If the browser reports NET::ERR_CERT_COMMON_NAME_INVALID, the SAN does not match the name you typed (see Appendix A).

> **Done when**
> - http://<StudentID>.f26 and https://<StudentID>.f26 both load the same website on the second device.
> - The certificate details match the table above.
> - Every check in Part H passes.
> - Your Student ID is visible on the website.

Deliverable: a working Apache2 deployment, demonstrated end to end in Video 1.

WHAT TO SHOW IN VIDEO 1

- ☐ Your full name and Student ID, at the very start.
- ☐ Apache2 installed and running.
- ☐ Your website files with their ownership and permissions (ls -l), and the website open in a browser.
- ☐ Your HTTP and HTTPS virtual host files, and the commands that enable them.
- ☐ The domain resolving to VM1 (the hosts entry and a ping), on VM1 and on the second device.
- ☐ The OpenSSL command that generates your certificate, and the verification output.
- ☐ The firewall status and the rules you added (or the statement that the firewall is inactive).
- ☐ The local tests from Part H.
- ☐ The second-device test of both addresses, including the certificate details in the browser.

Never display the contents of your private key.

6. Task 2: Automated Cross-System Backup with Email Notification (35 points)

Build a small but complete automation pipeline between two Linux systems that mirrors a real backup workflow used in production environments. Figure 2 shows how the three parts fit together.

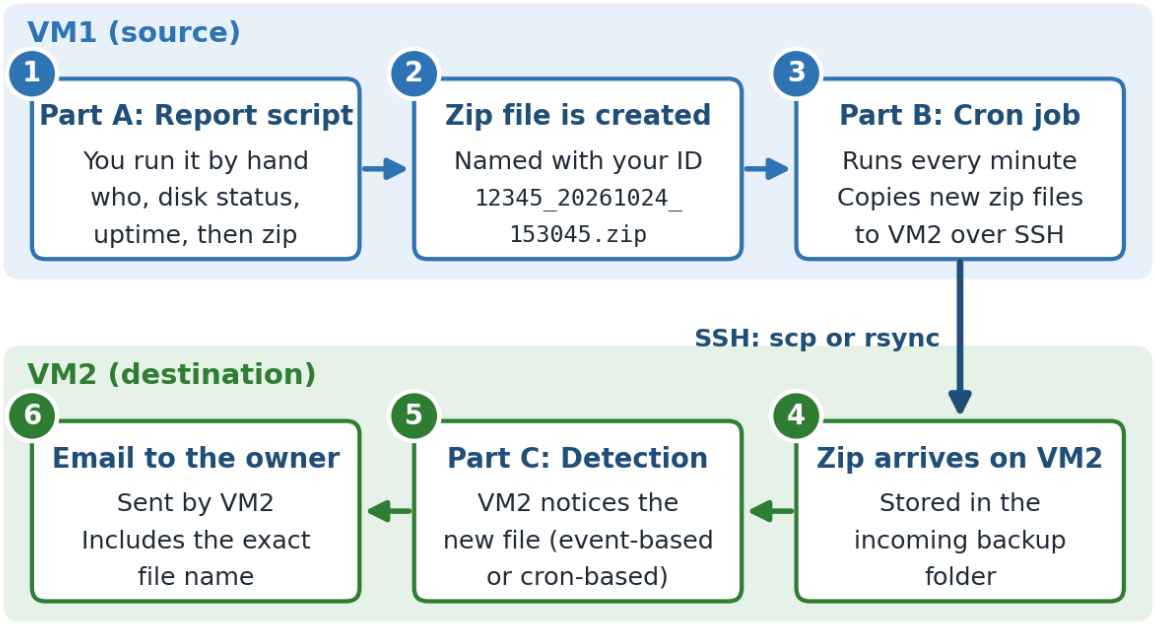


Figure 2: The Task 2 pipeline, from the manual report script to the email notification

- **Part A (VM1):** you run a Bash script by hand. It collects system information, saves it to a text file, and compresses it into a correctly named zip file.
- **Part B (VM1):** a cron job runs every minute and only transfers new zip files to VM2.
- **Part C (VM2):** VM2 detects the new file and emails the system owner (you) with the exact file name.

Task 2 Setup: SSH and Folders

You need two Linux systems: **VM1** (the source) and **VM2** (the destination), on the same network and able to reach each other. Configure SSH access from VM1 to VM2. Key-based authentication is strongly recommended so that the automated transfer can run without a password prompt.

1. Check connectivity in both directions with `ping -c 3 <other IP>`.
2. Make sure `openssh-server` is installed and running on VM2. Use a normal user account on VM2 as the destination account, not root.
3. On VM1, create an SSH key pair with no passphrase (`ssh-keygen`).
4. Copy the public key to your account on VM2 (`ssh-copy-id`).
5. Connect once by hand from VM1 (`ssh <user>@<VM2 IP>`) and accept the host key. Cron cannot answer that question later, so this first connection must be done manually.
6. Confirm that `ssh <user>@<VM2 IP>` now logs in without asking for a password.
7. Create the folders you will use. A simple layout is: on VM1, an **outgoing** folder for new zip files and a **sent** folder for files already transferred; on VM2, an **incoming** folder that receives the files.
8. If a firewall is active on VM2, allow SSH (`ufw allow 22/tcp`).

> **Why the key has no passphrase**
> Cron runs without you at the keyboard, so it cannot type a passphrase. Protect the key with file permissions instead (mode 600 on the private key, 700 on `~/.ssh`), and use it only for this exercise.

Part A: The Report Script (Runs Manually, on VM1)

Write a Bash script that a user runs manually on VM1. Each time it is run, it must:

1. Collect, at a minimum: **who is currently logged in**, the **current disk status**, and **how long the system has been up**.
2. Save this information into a **text file**.
3. Compress that file into a **zip archive** named with the exact pattern `StudentID_YYYYMMDD_HHMMSS.zip`, using your real Student ID and the precise timestamp at which the file was created.

GUIDANCE

- The script starts with `#!/bin/bash`, is made executable with `chmod +x`, and lives in a fixed folder (for example `~/scripts`). You run it from the terminal.
- Commands that provide the required information: `who`, `df -h`, and `uptime`. Adding the date, the host name, and your Student ID inside the report is good practice.
- Create the timestamp with `date` and the format `+%Y%m%d_%H%M%S`. Read the time **once**, store it in a variable, and use that variable in the file name, so the name and the time always agree.
- Save the zip in the outgoing folder on VM1 (the folder that Part B transfers from). You need the `zip` package for this.
- Every run must create a **new** zip with a new name. Wait at least one second between two runs, because the timestamp changes once per second.

EXAMPLE OF A CORRECT FILE NAME

Student ID 12345, created on 24 October 2026 at 15:30:45:

```
12345_20261024_153045.zip
```

SUGGESTED LAYOUT OF THE TEXT FILE

```
Student ID: 12345
Report created: 2026-10-24 15:30:45
Host: vm1

=== Logged-in users ===
(output of who)

=== Disk status ===
(output of df -h)

=== System uptime ===
(output of uptime)
```

CHECK IT

```
ls -l <outgoing folder>
unzip -l <zip file>
unzip -p <zip file>
```

The file name must match the pattern exactly, and the content must show all three required pieces of information.

Part B: The Auto-Backup Cron Job (Runs Automatically, on VM1)

1. Create a cron job on VM1 that runs **every 1 minute** (schedule `* * * * *`).
2. Each time it runs, it must automatically transfer any **newly created zip file(s)** from VM1 to VM2 (for example using `scp` or `rsync` over SSH).
3. The cron job **only transfers files**. It must not generate the report itself. That is the job of the manual script in Part A.

> **Important**
> Do not call the report script from cron. A cron job that creates reports does not meet the Part B requirement.

GUIDANCE

- Put the transfer commands in a small script (for example `~/scripts/send_backups.sh`) and call that script from cron. This keeps the cron line short and avoids problems with the `%` character, which must be written as `\%` inside a crontab line.
- Use **absolute paths** everywhere. Cron starts with a minimal environment and does not use your terminal's working directory or PATH.
- Send each file **only once**. If the job copies every zip every minute, VM2 receives the same files again and again, and you may get repeated emails. Choose one pattern: (1) use `rsync`, which skips files that already exist unchanged on VM2; or (2) use `scp` and, after a successful transfer, move the file from the outgoing folder to the sent folder.
- Append the job's output to a log file (`>> logfile 2>&1`) so that you can see what happened.
- Edit the schedule with `crontab -e` as the same user who owns the SSH key, and confirm it with `crontab -l`.

CHECK IT

Create a zip with the Part A script, wait up to one minute, and confirm that the file appears in the incoming folder on VM2 (and, if you use pattern 2, in the sent folder on VM1). If it does not, read your log file.

Part C: Arrival Notification (on VM2)

1. **Detect**, on VM2, when a new backup file has arrived.
2. **Automatically send an email** to the system owner (you) to notify them that a new backup file has been transferred.
3. The email must include the **exact name** of the file that was received.

CHOOSE A DETECTION METHOD

Method	How it works
Event-based	A watcher such as `inotifywait` (package `inotify-tools`) monitors the incoming folder. Trigger on a completed file (events `close_write` or `moved_to`), not on the first `create` event, which fires before the file is complete.
Time-based	A cron job on VM2 runs every minute, compares the incoming folder with a record of the files already reported, and sends an email only for files that are not yet in the record.

Use the method that was demonstrated in class. Either way, send **exactly one email per new file**.

EMAIL CONTENT AND MAIL SETUP

- Put the exact file name in the subject and in the body, for example the subject `New backup received: 12345_20261024_153045.zip`. The time of arrival and the name of the source VM are useful extras.
- Install and configure your mail tool on **VM2**. Most home and campus networks do not allow a VM to deliver mail directly (port 25 is commonly blocked and direct mail is often rejected). Configure your mail tool to send through an authenticated SMTP relay, for example a mailbox provider's SMTP server on port 587 with TLS. Follow your provider's current instructions, because some providers require an app password instead of your normal password.
- Test the mail setup **on its own** before you connect it to the detection step: send a plain test message from the VM2 terminal and confirm that it arrives (check the spam folder).
- A cron job or a background watcher starts with a minimal environment. Use absolute paths for the mail command and for every file.

Protect your credentials

Never show your real mailbox password or app password in a video or in your Task 3 documentation. Store credentials in a file that only your user can read (mode 600), cover or blur them while recording, and revoke the app password after grading.

Rehearse the Whole Chain Before Recording

1. On VM1, run the report script by hand and note the name of the zip file.
2. Within one minute, the zip file appears in the incoming folder on VM2.
3. VM2 detects the new file and sends the email.
4. The email arrives in your mailbox, and its subject or body contains the exact file name from step 1.
5. Repeat once to prove that every run produces its own zip file with a different name and its own email.

Done when

- The manual script creates a zip named `StudentID_YYYYMMDD_HHMMSS.zip` that contains who is logged in, the disk status, and the uptime.
- The cron job runs every minute and transfers only new zip files to VM2 without a password prompt.
- VM2 sends an email that contains the exact name of each received file.

Deliverable: a demonstrated end-to-end run in Video 2, showing the manual script producing a correctly named zip on VM1, the cron job picking it up within a minute, and the resulting email notification arriving.

SUGGESTED POINTS TO EXPLAIN IN YOUR NARRATION

- Why key-based SSH authentication is used for an automated transfer.
- How the script builds the file name from your Student ID and the timestamp.
- What the schedule `* * * * *` means, and why the cron job uses absolute paths.
- How your solution avoids sending the same file, or the same email, twice.
- How VM2 detects a new file, and how the email receives the exact file name.

WHAT TO SHOW IN VIDEO 2

- ☐ Your full name and Student ID, at the very start.
- ☐ Both systems, with the user name and host name visible in the terminal prompt so that the viewer can tell VM1 from VM2. Show their IP addresses and a successful ping.
- ☐ The SSH login from VM1 to VM2 without a password.
- ☐ The code of the report script, then a live run. Show the zip file name and its content (`unzip -l` and `unzip -p`).
- ☐ The cron entry (`crontab -l`) and the transfer script.
- ☐ The zip file arriving on VM2 within a minute (show the folder on VM2 live).
- ☐ The email arriving in your mailbox, with the exact file name.

7. Task 3: Documentation Server with NGINX (20 points)

Deploy a second web server, this time using NGINX, to host professional, tutorial-style documentation of the work you did in Tasks 1 and 2.

Task 3 Requirements

1. **Install and configure NGINX** on a Linux machine or VM. If you reuse the same machine as Task 1, run NGINX on different ports, because 80 and 443 are already in use by Apache2.
2. **Build and host a documentation page** (or a small site) written like a tutorial that someone else could follow. It must cover, step by step, the content listed in the table below.
3. Make it **detailed enough that another student could follow it and reproduce your setup**. Include the commands you ran and the key configuration lines you changed, with a short explanation of what each step does.
4. Your **Student ID must be clearly visible** on the documentation page.
5. **Confirm that the documentation site is reachable** from another device on the same LAN.

Where to Run NGINX

Option	Details
Option A (simplest)	Install NGINX on **VM2** and use the default port 80. Apache2 is not installed there, so there is no conflict. From the second device, open `http://<VM2 IP>`.
Option B	Install NGINX on **VM1** and use other ports, for example 8080. Apache2 must keep ports 80 and 443. From the second device, open `http://<VM1 IP>:8080`.

A domain name is not required for Task 3. You may use the IP address, or add another hosts entry if you prefer a name.

Deploy NGINX

1. Install NGINX and confirm that it is running (`systemctl is-active nginx`).
2. Create a folder for your documentation (for example `/var/www/12345-docs`) and place your pages in it.
3. Create a server block with the `listen`, `server_name`, `root`, and `index` directives in `/etc/nginx/sites-available`, and enable it with a symbolic link in `sites-enabled`. Remove or disable the default site if it conflicts.
4. Test the syntax with `sudo nginx -t`, then reload with `sudo systemctl reload nginx`.
5. If a firewall is active, allow the port you chose.
6. Test locally with `curl -I http://localhost` (add the port number for Option B), then from the second device.

What Your Documentation Must Contain

Write the page as a tutorial. A classmate who reads only your page should be able to rebuild your setup.

Part	Cover step by step
Task 1: Apache2 over HTTP and HTTPS	• Installing Apache2, creating the site folder, and setting permissions • The HTTP virtual host and the domain (hosts file) • Generating the self-signed certificate with OpenSSL, and verifying it • Enabling the SSL module and the HTTPS virtual host • The firewall rules, the local tests, and the second-device test

Part	Cover step by step
Task 2: Backup automation	• Connecting VM1 and VM2: network check, SSH key, first login • The report script (full code, with an explanation) • The cron job (the cron line and the transfer script) • VM2 detection and the email notification (mail setup with secrets removed) • Testing the whole chain

RECOMMENDED FORMAT FOR EACH STEP

Goal, then the command or commands in a monospaced block, then the key configuration lines, then a short explanation, then the expected result.

Rules for the documentation page

- Your Student ID must be clearly visible on the page, for example in the title and in the header.
- Never include your private key, passwords, or app passwords in the documentation.

Done when

- NGINX is running and serves your documentation page.
- The page is reachable from another device on the same LAN.
- The page covers Task 1 and Task 2 step by step, with commands, key configuration lines, and explanations.
- Your Student ID is clearly visible on the page.

Deliverable: a working NGINX deployment hosting your Task 1 and Task 2 documentation, demonstrated in Video 3.

SUGGESTED POINTS TO EXPLAIN IN YOUR NARRATION

- What a server block is, and how NGINX knows which folder to serve (the `root` directive).
- Why NGINX needs different ports when it shares a machine with Apache2, or why VM2 has no conflict.
- Why you test the syntax with `nginx -t` before you reload the service.

WHAT TO SHOW IN VIDEO 3

- ☐ Your full name and Student ID, at the very start.
- ☐ NGINX installed and running, and your server block configuration.
- ☐ The syntax test (`nginx -t`) and the reload.
- ☐ The documentation page opened from the second device, with your Student ID visible.
- ☐ A scroll through the page that shows the Task 1 and Task 2 sections.

8. Submission Requirements

You will submit **three (3) separate video tutorials**, one per task.

Video	What it covers	File name
Video 1	Task 1 walkthrough (Apache2, HTTP and HTTPS, webpage)	`StudentID_Task1.mp4`
Video 2	Task 2 walkthrough (backup automation, cron, email notification)	`StudentID_Task2.mp4`
Video 3	Task 3 walkthrough (NGINX documentation site)	`StudentID_Task3.mp4`

Every Video Must

- **Open with you clearly stating your full name and Student ID**, on camera or on screen.

- **Show your terminal and the commands you run at each step**, not only the final result.
- **Show the final working result live** (the page loading in a browser, the email arriving, and so on), not as a static screenshot.
- **Be narrated or clearly captioned**, so that a viewer can follow what you are doing and why.

> **Example opening sentence**
> "Hello, my name is <full name> and my Student ID is <Student ID>. This is Video 1: Task 1, the Apache2 web server."

Recording Tips

- Increase the terminal font size so that commands are readable in the recording, and use a high-contrast color scheme.
- Show a terminal prompt with your user name and host name, so that the viewer can tell VM1, VM2, and the second device apart.
- Run `clear` between steps, and pause for two or three seconds on important results so that they can be read.
- Explain **why** you run each command, not only what it does. This is what "narrated" means.
- Test your microphone and play back a short recording before you start.
- Rehearse once before the final recording. Never show passwords or the contents of your private key.

File Naming and Submission

- Name your files `StudentID_Task1.mp4`, `StudentID_Task2.mp4`, and `StudentID_Task3.mp4`, using your real Student ID. Use MP4 unless your institution requires another video format.
- Submit via: [**Google Form**].
- **Deadline: Saturday, 24 October 2026, 11:59 PM.** See the late submission policy in Section 10.

> **Before you upload**
> ☐ I played each video from start to end on another device.
> ☐ The first seconds show or state my full name and Student ID.
> ☐ Commands and results are readable, and the narration is audible.

9. Academic Integrity and Student ID Policy

This project is built so that substituting a classmate's work is easy to catch. Your Student ID must appear, visibly and correctly, in **every** one of the following places.

Where	Task	What must be visible
The SSL certificate you generate	Task 1	Your Student ID in the Common Name (CN) field. The Organization field is AUCA, and the domain is in the SAN.
The webpage content	Task 1	Your Student ID displayed on the website hosted by Apache2.
The domain name and both virtual hosts	Task 1	`<StudentID>.f26` used as the server name and in the web address.
The backup zip file name	Task 2	`StudentID_YYYYMMDD_HHMMSS.zip`, with your real Student ID.
The documentation page	Task 3	Your Student ID clearly visible on the page hosted by NGINX.
The start of every video	Tasks 1 to 3	Your full name and Student ID, spoken and/or shown on screen at the very start.

IN ADDITION

- Set up your environment the same way it was demonstrated in class. Unusual shortcuts, unfamiliar tools, or configurations that do not match what was taught are a flag for independent review.
- Work must be done individually. Sharing scripts, certificates, configuration files, or recordings between students is a violation for everyone involved, not just the person who copied.

11. Evaluation Criteria

Component	Points	What is being checked
Task 1: Apache2 HTTP/HTTPS	30	• Correct installation and configuration • A valid self-signed certificate with your Student ID • Both HTTP and HTTPS reachable from the LAN • The webpage contains the required summary content
Task 2: Backup automation	35	• The script captures all three required pieces of information • The zip file is named correctly • Cron runs every minute and transfers successfully • VM2 sends an email with the correct file name
Task 3: NGINX documentation	20	• NGINX is correctly deployed • The documentation is clear, accurate, and reproducible • It covers both Task 1 and Task 2 • It is reachable from the LAN
Video submission and ID compliance	15	• All 3 videos are submitted on time • Each video shows the full process rather than only the result • Your Student ID is visible or stated everywhere it is required
Total	**100**	

Mistakes to avoid

- Your Student ID is missing from one of the required places (Section 9).
- The certificate has no SAN, or the Organization, Course, or Student ID values are wrong.
- HTTPS is configured but never tested from the second device.
- The cron job generates the report instead of only transferring files, or it re-sends old files every minute.
- The zip file name does not follow `StudentID_YYYYMMDD_HHMMSS.zip` exactly.
- The email does not contain the exact file name.
- The documentation does not cover both tasks, or it is not detailed enough to reproduce your setup.
- A video shows only the final result, or it does not open with your name and Student ID.

12. Final Checklist Before You Submit

TASK 1

- ☐ Apache2 serves my site on both HTTP and HTTPS, and both addresses are reachable from another device on the LAN.
- ☐ My SSL certificate includes my Student ID (CN), AUCA (O), Linux-F26 (OU), my domain (SAN), and about one year of validity.
- ☐ My webpage includes my learning summary (commands, shell scripting, users and permissions) and my Student ID.
- ☐ My firewall is not disabled, and ports 80 and 443 are allowed if it is active.

TASK 2

- ☐ My manual script on VM1 correctly captures who is logged in, the disk status, and the uptime.
- ☐ My zip file is named `StudentID_YYYYMMDD_HHMMSS.zip`.
- ☐ My cron job on VM1 transfers new zip files to VM2 every minute, and it does not create reports.
- ☐ VM2 emails the owner with the correct file name when a new backup arrives.

TASK 3

- ☐ NGINX hosts a clear, step-by-step write-up of Task 1 and Task 2, reachable from the LAN.
- ☐ My documentation page includes my Student ID.

VIDEOS AND SUBMISSION

- ☐ I have 3 videos, correctly named, each opening with my name and Student ID.
- ☐ No video shows a password, an app password, or the contents of my private key.
- ☐ All files are submitted before 24 October 2026, 11:59 PM.

"Dream big and work hard to achieve your goals. Stay strong and keep fighting for your dreams. Do not let anything discourage you. The future looks bright."

Good luck!